店智 Copilot 技术文档

面向客户的电商智能客服 Agent：售前查商品推荐、售后查订单退款、投诉生成工单

项目	内容
文档版本	**v1.2 • 2026-10-28**
参赛团队	邢博、刘庆昊、孔克铮
学校	河北传媒学院
专业	人工智能
文档范围	正文 8 章 + 附录（API、环境变量、复现命令、版本栈）
代码事实基线	43 篇语料 · 12 段场景 · 20 单种子订单 · 4 个入口

摘要

店智 Copilot 是面向 1 至 5 人电商小团队的浏览器工作台。它把运营者一天里最常打断工作的四件事收进同一个对话框：看数据、查问题、处理售后、写文案。系统由 Next.js 16 前端与 FastAPI AI 接入层组成，前端不直连任何模型；文本推理走 DeepSeek Chat，商品图理解走 GLM-4V-Flash，检索由 jieba BM25 与 fastembed BGE-small-zh 向量经 RRF 融合完成。

项目已落地 4 个入口（工作台、对话、数据、知识库）、43 篇知识库语料、12 段可复现场景、20 单种子订单、10 个 Agent 工具与 4 个 MCP 工具。Agent 采用手写 ReAct 循环，单轮最多 5 步；退款、改地址、拦截物流、退换货四类敏感操作统一走 validate → 用户确认 → execute 的两段式门控，模型不直接动钱。检索自测 top-3 相关率 90%，交付冒烟 11/11，前端 tsc、eslint、build 全部通过。

关键词：AI 工作台；RAG；Agent；ReAct；MCP；两段式门控；可溯源引用

1 项目概述

1.1 产品定位

店智 Copilot 的定位是**面向客户的电商智能客服 Agent**，形态是浏览器打开即用的对话式客服。左侧两个入口：`/chat` 客服对话、`/kb` 知识库，根路径 / 重定向到对话。基于 LangGraph 多 Agent 编排：Router 自动分流售前/售后/投诉/澄清，售前售后各自隔离的工具集，终审防幻觉。

它不是通用聊天机器人，不是传统工单系统，也不是单纯的数据看板。聊天机器人只给建议，工单系统只排队，看板只展示数字；店智 Copilot 把「看见问题」和「把问题办掉」放在同一条对话链路里。核心主张是**咨询即办理**：对话里说清楚诉求，Agent 查订单、查物流、跑校验、给预览，用户回一句「确认」，操作落库并返回回执。

1.2 目标用户与两类场景

系统服务两类用户：**电商店铺的客户**（下单前咨询商品、比价选款；下单后查物流、办退款退货、投诉）与**商家**（在知识库维护商品知识与售后政策，Agent 据此接客）。

客户问题归为两类场景：**售前**（商品咨询、智能推荐、查库存）与**售后**（查订单、退款退货、投诉升级工单）。LangGraph Router 自动分流，售前售后各自隔离工具集，终审统一审查防幻觉。

两类场景共享同一套会话、工具与引用机制。查完订单可以直接退款，退款被拒可以接着问规则，规则答完可以把新规则补进知识库，用户不用切换心智模型。

1.3 事实基线

本文所有数字均可在仓库内复核，未使用推测值或行业均值。

项	当前值
知识库与场景	**43 篇**语料，覆盖 10 个类目；**12 段**场景案例
数据与入口	**20 单**种子订单；**4 个**页面入口加 / 重定向
Agent 与 MCP	**10 个**工具、单轮最多 **5 步**；MCP 暴露 **4 个**
质量门禁	检索 top-3 **90%**；交付冒烟 **11/11**
关键样例	001 退 ¥80 剩 **¥79.8**；009 圆通 **YT9182736455102**

1.4 与同类方案的边界

与通用聊天机器人相比，店智 Copilot 的回答有 `[n]` 角标可回查原文，工具调用以步骤卡展示，敏感操作有后端门控；与传统工单系统相比，它不把问题推给排队，而是在对话内完成退款、改址、拦截与退换。边界同样明确：`data_query` 查本

地 30 天种子库，售后操作作用于种子订单，不接平台真实授权。

---

2 需求分析

2.1 行业痛点

电商客户的困难不在于「没有地方问」，而在于问题分散在不同入口，且每个都要重新描述一遍。

规则查询难。 平台规则分散在商家后台、公告与处罚记录里，更新频繁。保证金、处罚标准、响应时效、退款率后果这类问题靠经验记忆容易踩坑。通用模型的回答看起来合理，却无法核对依据，规则一旦更新，模型记忆反而成为风险。

数据解读门槛高。 后台能给数字，不给判断。退款率 3.13% 是高还是低，哪几单把指标拉上去，先修转化还是先清售后，需要把指标、订单、售后单放在一起看。运营者往往知道「不对劲」，但说不出「哪里不对」。

文案与售后重复消耗。 标题、卖点、短描述格式固定、产量大；退款、改地址、拦截物流、退换货步骤固定、打断频繁。两类工作都不难，但都要在多个页面之间复制粘贴，容易漏步骤。

项目因此把「可查数、可引用、可办理、过程可见」作为需求起点。

2.2 三类用户任务

把四类工作按认知负担重新分组，可以得到三类任务。三类任务对应两套 AI 模式，也对应不同的成功判据。

认知型任务（看数据、查问题）走 RAG 或 Agent：`knowledge`、`general` 留在 RAG，`data`、`policy`、`copywriting` 切到 Agent，成功判据是数字真实、归因有据、建议可执行。办理型任务（处理售后）必须走 Agent 工具与两段式门控，要求不误操作、可审计、重复提交不重复扣款。生产型任务（写文案、合规自查）由 `copywriting` 工具或 GLM-4V 识图完成，要求结构完整、长度可控、可直接编辑。

2.3 功能需求

模块	需求与验收
客服对话	LangGraph 多 Agent 编排，Router 分流售前/售后/投诉/澄清，思维链过程可视化
售前	商品咨询、智能推荐、查库存；[n] 角标引用可溯源
售后	订单与物流查询；退款、改址、拦截、退换四类两段式门控；投诉升级生成工单
知识库	43 篇检索、[n] 角标、切片编辑保存即生效
多模态与开放	商品图文案、语音输入；MCP 暴露 4 个工具
### 2.4 非功能需求	

编号	类别	要求	实现与证据
NFR-01	可溯源	每条知识型回答能回到原文	system 注入带编号片段，`/v1/rag/search` 同源回查
NFR-02	安全	模型不得直接执行资金操作	后端校验 + 一次性令牌 + 用户确认 + 幂等
NFR-03	可控	单轮工具调用不得无限循环	`MAX_STEPS=5`，超限返回明确错误
NFR-04	可用	缺密钥、缺索引、上游故障时不白屏；回答逐字呈现	中文错误 JSON + 降级；SSE `step` / `delta` / `done`
NFR-05	可维护	新增工具不改循环；索引与种子数据可复现	全局注册表；`pipeline build`、`seed_store`
NFR-06	成本	嵌入与语音不产生额外 API 费用	fastembed 本地嵌入，Web Speech 浏览器识别

2.5 约束与不做的事

比赛规则要求后端服务仅作为 AI 能力的接入层，因此所有模型调用、密钥读取、检索与工具执行都放在 FastAPI，前端只做界面与协议转换。项目按单机部署设计，不引入账号体系、多租户与权限模型。明确不做的功能包括真实平台授权与资金

流、RPA 自动回复、多 Agent 编排、长期记忆、LoRA 微调、暗色模式与 TTS；第 8 章把值得继续做的部分列入路线图，并标注「未实现」。

---

3 系统架构设计

3.1 分层架构

系统分四层：浏览器界面、Next.js 同源代理、FastAPI AI 接入层、外部模型与本地索引。前端永远不直连模型 API，浏览器也永远不直连 8000 端口；所有 `/api/*` 请求经 Next.js 同源代理转发到 FastAPI。

```
浏览器  Next.js 16  /chat /kb
  │  流式 Markdown · Agent 步骤卡 · [n] 引用 · 图片/语音
  ▼  同源 /api/* 代理（浏览器不直连模型）
FastAPI  backend/app/    AI 能力唯一接入层
  ├─ api/        chat · agent · vision · sessions · refund · kb
  ├─ agent/      ReAct 循环（≤5 步）+ 10 工具注册表
  ├─ rag/        分块 → 嵌入 → LanceDB → BM25+向量 RRF
  ├─ store       SQLite 会话库 + 30 天经营种子库
  └─ mcp_server/ 独立进程（stdio，4 工具，复用注册表）
  ▼
DeepSeek Chat（文本 + function calling）
GLM-4V-Flash（商品图，可选）
```

分层带来的直接好处是密钥只存在于 `backend/.env`。浏览器拿不到 `DEEPSEEK_API_KEY`，前端代码里也没有任何模型端点。即使有人打开开发者工具，也只能看到同源 `/api/*` 请求。

3.2 一次请求的数据流

客户请求经同源代理进入 FastAPI，由 LangGraph 编排分流到售前/售后/投诉/澄清。下面的流程覆盖主路径与工具回填分支。

```
用户输入
  → Next.js /api/chat 或 /api/agent（同源代理）
  → FastAPI /v1/chat/completions 或 /v1/agent/chat
  → 意图分流
       knowledge / general  → RAG
       data / policy / copywriting → Agent
  → RAG：检索 top-4 → 注入带编号 system → DeepSeek 流式
  → Agent：DeepSeek 返回 tool_calls → 执行工具
        → SSE step / step_result → 回填 tool message
        → 下一轮（最多 5 步）
  → SSE delta 流式文本 → done（含 citations）
  → 前端渲染步骤卡、订单卡、[n] 角标
```

RAG 检索在请求进入上游前完成，命中片段前置到 system；Agent 每轮非流式调用以完整拿到 `tool_calls`，最终答案才按 8 字符块通过 `delta` 推送。

3.3 技术选型

层	选型	版本	选择理由
前端	Next.js + React + TypeScript + Tailwind + Vercel AI SDK	16.4.2 / 19.3.1 / 6.0.3 / 3.4.19 / 7.0.148	App Router 与同源代理成熟；`streamText`、`useChat` 直接消费 SSE
后端	Python + FastAPI + uvicorn	3.12 / 0.115+ / 0.32+	与 RAG、Agent、MCP 同语言，接入层边界清晰
AI 能力	DeepSeek Chat；GLM-4V-Flash（可选）	OpenAI 兼容接口	中文与 function calling 稳定；视觉走免费档
检索	fastembed BGE-small-zh + LanceDB + jieba BM25	512 维 / 0.40+ / k1=1.5	零密钥本地嵌入，嵌入式向量库，双通道同进程融合

层	选型	版本	选择理由
协议与存储	FastMCP stdio；SQLite	1.x；stdlib	Inspector 可测；会话与种子库零部署

3.4 关键架构决策

编号	决策	理由	代价
D1	手写 ReAct，不用 LangChain/LangGraph	10 个工具、单一模型，框架收益小于抽象成本；SSE 协议可自定	自维护循环与步数上限
D2	backend/app/ 分层包结构	api / agent / rag / store 职责分离，加工具不污染入口	文件数与导入路径变长
D3	BM25 + 向量 RRF 融合	规则依赖关键词，口语依赖语义，单通道会漏	两路候选与融合需自实现
D4	MCP 独立进程 stdio	不与 FastAPI 端口冲突，Inspector 可直接连	进程生命周期需单独管理
D5	Agent 步骤可视化	工具调用是信任来源，也是体验差异	前端需解析结构化 summary
D6	评测 v2 按引用有据性评分	裸模型常识题已很强，优势须在防编造与可溯源维度评	需设计 15 题与裁判口径

4 AI 技术方案

4.1 LLM 与技术组件选型理由

选型的第一原则是**能力边界与接入层边界一致。**文本、视觉、嵌入三类能力分别选型，不追求一个模型包打天下；每一类都保留可替换实现，避免被单一供应商锁死。

文本模型选 DeepSeek Chat。它提供 OpenAI 兼容接口，`tools` 参数与 `tool_calls` 返回结构稳定，适合手写 ReAct 循环；中文表达与技术术语准确，退款、物流、平台规则这类语境不需要额外微调；按 token 计费，演示与评测成本可控。备选是 GLM-4-Flash 与 GPT-4o-mini，前者作为免费兜底，后者成本更高且对本地化规则无额外优势。

视觉模型选 GLM-4V-Flash。DeepSeek 文本模型没有视觉能力，商品图必须先转成属性文本。GLM-4V-Flash 有免费档，接口格式与 OpenAI 兼容，返回 JSON 后可直接进入 `copywriting` 工具。它是可选链路，未配置 `VISION_API_KEY` 时返回 400 中文提示，不影响文本与 Agent 主链路。

嵌入选 fastembed 的 BGE-small-zh-v1.5，512 维，本地运行。理由有三条：不需要嵌入 API 密钥，评审拉下仓库即可重建索引；中文语义检索质量足够支撑 43 篇语料的 top-4 召回；离线可复现，不受网络与限流影响。需要更大上下文时可用 `EMBED_PROVIDER=siliconflow` 切到 BGE-M3，代价是引入密钥与网络依赖。

向量库选 LanceDB。演示规模只有几十个切片，Milvus、Postgres/pgvector、Redis 都带来额外部署与运维成本。LanceDB 以嵌入式方式与 Python 同进程，索引落盘为 `data/index/local/chunks.lance`，重建命令一条，评审可复现。

关键词通道选 jieba 加自实现 BM25，而不是 Elasticsearch。规则条文里「保证金」「体验分」「直通车」这类专有词，分词后命中很直接；BM25 的 k1 与 b 可调，代码不足百行，和向量通道共享同一批候选，RRF 融合不需要跨服务调用。

编排层不引 LangChain/LangGraph，不引 Dify/FastGPT。低代码平台会让「自研 Agent」这一叙事失效，框架则把工具循环、事件流与错误处理包进黑盒。当前工具数量与步数上限都可控，手写循环反而更容易讲清楚每一次工具调用发生在什么时候、参数是什么、结果如何回填。

组件	备选	不选的原因
DeepSeek Chat	GPT-4o-mini、GLM-4-Flash	成本更高，或 function calling 与中文稳定性未验证
GLM-4V-Flash	GPT-4V	需要额外付费，商品图任务不需要更强视觉推理
fastembed 本地嵌入	OpenAI Embedding、BGE-M3 云	引入密钥与网络依赖，43 篇语料本地已够
LanceDB	FAISS、Chroma、Milvus	需额外服务或缺少持久化，演示规模过重

组件	备选	不选的原因
手写 ReAct	LangChain、LangGraph、Dify	黑盒削弱可控性与自研叙事
FastMCP stdio	HTTP MCP	本地进程更简单，Inspector 直接可用

4.2 Prompt 设计

Prompt 分四类资产：RAG 注入模板、Agent 系统提示、意图分类提示、视觉文案提示。四类都遵守同一组约束：角色明确、输出结构固定、缺依据时允许说「不知道」、不把内部实现细节暴露给用户。

RAG 注入模板在检索命中后前置为 system 消息。片段带编号、标题、分类与来源，回答要求句末标注角标，检索不足时明确说「知识库未覆盖」。

```
以下是内部知识库的检索结果，仅用于参考回答：

[1] 《DSR 评分规则解读》（分类：平台规则，来源：知识库）
<片段原文>

[2] 《体验分与店铺权益》（分类：账号与体验分，来源：知识库）
<片段原文>

要求：引用依据时在句末标注 [1][2] 形式的角标；
如果检索结果不足以回答问题，请明确说明"知识库未覆盖"，不要编造。
```

Agent 系统提示只保留行为原则，不写具体业务规则。业务规则放在后端工具里，模型无法通过提示词绕过校验。

提示词	关键约束	设计意图
Agent system	先查工具再建议；数字与规则带角标；给可执行下一步	把「查证」放在「表达」之前
情绪前缀	非平静情绪在用户消息前加「先安抚」提示	投诉与焦虑场景先处理情绪再办事
意图分类	五类意图 + 路由目标 + 实体 + 情绪，温度 0	输出可解析，路由稳定，失败降级 general
视觉文案	标题 30 字内、3 至 5 个卖点、一句短描述、JSON 字段固定	直接把模型输出接到 UI 与 copywriting
RAG 注入	编号片段 + 角标要求 + 未覆盖声明	让引用可回查，让拒答有出口

意图分类是唯一使用 `temperature=0` 的调用。它不需要创造力，只需要稳定复现同一路由。分类失败时不抛错，降级为 `general`，由 RAG 兜底；这条降级路径保证意图服务故障不会让整个对话不可用。

4.3 Agent 与 ReAct 循环

Agent 内核是一个显式的 `for step in range(MAX_STEPS)` 循环。每轮把当前消息列表与 10 个工具的 JSON Schema 发给 DeepSeek；有 `tool_calls` 就逐个执行、追加 `tool` 消息、进入下一轮；没有 `tool_calls` 就把 `content` 切成小块，以 `delta` 事件推给前端。

```
messages = [system, ...history, user]
for step in 1..5:
    resp = DeepSeek(messages, tools=10 个 schema, stream=false)
    if resp.tool_calls:
        for call in resp.tool_calls:
            emit step(call.name, call.args)
            result = REGISTRY[call.name].call(call.args)
            emit step_result(call.name, summary(result))
            messages.append(tool_message(call.id, result))
        continue
    emit delta(resp.content, 每块 8 字符)
    emit done(citations)
    return
emit error("已达最大工具调用步数 5")
```

SSE 事件	载荷	前端行为
step	{tool, args}	新建步骤卡，显示工具名与参数
step_result	{tool, summary}	展开卡片，显示结果 JSON 或摘要
delta	{text}	追加到当前消息，逐字渲染
done	{citations}	结束加载态，挂载引用角标
error	{message}	内联错误提示，保留已输出内容

10 个工具按职责分三组：知识组、数据组、办理组。知识组复用 RAG；数据组走 DataProvider；办理组统一采用两段式协议。

工具	参数	返回	用途
kb_search	query	top-3 片段（标题、分类、正文 400 字）	知识库检索，回答引用来源
data_query	query	GMV、订单、访客、转化率、退款率、热销品、流量来源	经营快照，带 source 标明数据来源
copywriting	product_name, selling_points	title、short_desc、bullet_points	商品文案生成
policy_check	scenario	risk_level、suggestion	场景合规检查，内部走检索
product_lookup	keyword	价格、库存、上下架、低库存提示	商品与库存查询
order_lookup	order_id?, status?	orders[] + aftersales[]	订单与售后状态查询
refund_flow	action, order_id, amount?, token?	preview、token、receipt	两段式退款
modify_address	action, order_id, new_address?, token?	preview、token、receipt	两段式改收货地址
intercept_logistics	action, order_id, token?	preview、token、receipt	两段式拦截物流
return_exchange	action, order_id, return_type?, reason?, token?	preview、token、receipt	两段式退换货建单

工具异常不抛给前端。未知工具名、参数缺失、后端业务拒绝都转成可读字符串回填给模型，模型可以据此改写回答或换一条路径。这条约定让单次工具失败不会中断整条 SSE。

实测案例：「查店铺今日数据并给建议」。模型先调一次 data_query 拿总体指标，再调一次 data_query 拿热销品与流量来源，然后输出 138 段流式文本，形成数据表、6 条解读与优先动作清单。两次调用是模型自己决定的，不是代码写死的编排。

4.4 RAG 检索与 90% 评测

4.4.1 语料与索引管线

语料是 43 篇 Markdown，覆盖 10 个类目。每篇先解析 frontmatter，再做文本清洗，然后按段落累积到 800 字符切块，块间重叠 200 字符。重叠的作用是避免一条规则被切断后，前半句在上一块、后半句在下一块，检索只命中半条。

```
原始 .md
  → frontmatter 解析（title / source / url / category）
  → 清洗（空行、重复标点、页眉页脚）
  → 分块（目标 800 字符，重叠 200）
  → fastembed BGE-small-zh-v1.5（512 维，batch=32）
  → LanceDB 持久化（data/index/local/chunks.lance）
```

索引构建是幂等的。python -m backend.app.rag.pipeline build 每次从 data/raw 全量重跑，覆盖同名表。评审可以删掉 data/index 后从零重建，不依赖任何预置二进制。

4.4.2 混合检索

检索有两条通道，各自取前 2k 个候选，再用 RRF 融合。

向量通道把查询嵌入后做余弦相似度检索。它擅长处理口语化表达，比如「客户老拿这个说事」与「DSR 评分」之间的语义关联。关键词通道用 jieba 分词，带字符 bigram 兜底，自实现 BM25，参数 k1=1.5、b=0.75。它擅长专有词与数值型条文，比如「保证金」「7 天售后期」「直通车」。

融合公式为 `score(d) = Σ 1 / (60 + rank_i(d))`，k 取 60。RRF 不比较两路分数的绝对值，只比较排名，避免向量相似度与 BM25 分数量纲不一致的问题。默认返回 top-4；`kb_search` 工具只取 top-3，减少 tool message 长度。

4.4.3 生成端注入

命中片段按 RRF 顺序编号，注入 system。编号顺序与 `/v1/rag/search` 返回顺序完全一致，这是引用能回查的前提。请求体带 `rag=false` 时跳过注入，评测脚本用这个开关做有 RAG 与无 RAG 的对照。

4.4.4 评测结果

评测分两版。v1 是 20 题通用对比，0/1/2 分制，满分 40；v2 是 15 题引用有据性评分，要求「正确 + 有引用 + 引用与原文一致」，满分 30。

版本	组别	总分	均分	说明
v1（20 题）	有 RAG	26 / 40	1.30	5 题因上游限流记为 -1，未计入有效对比
v1（20 题）	无 RAG	23 / 40	1.15	通用常识题裸模型已能答对，差距有限
v2（15 题）	有 RAG	29 / 30	1.93	run3，裁判上下文与生成端对齐
v2（15 题）	无 RAG	30 / 30	2.00	仅 Q15 语料空白题裸模型靠常识得分

v2 迭代过三轮，过程比结论更有信息量。

轮次	改动	有 RAG	无 RAG	失败题
run1	基线，裁判只读片段前 300 字	27	30	Q3 标题字符数、Q5 FABE、Q11 退款率
run2	新增 3 篇语料，语料达 **43 篇**	28	29	Q5 裁判误伤、Q15 随机波动
run3	裁判上下文改为完整原文，与生成端同序	**29**	**30**	仅 Q15

run1 的 Q3 与 Q11 是真实语料空白：模型诚实说「知识库未覆盖」，裁判按「无确切答案」给 1 分。补入标题字符限制与退款率后果两篇后，两题稳定拿到 2 分。这说明补语料的有效方向是补**数值型规则原文**，不是补方法论文章。

Q5 是评测 bug，不是产品 bug。生成端拿到完整 chunk，裁判端只拿到前 300 字，FABE 的 3C 认证示例恰好在 300 字之后，裁判看不到模型实际看到的内容，把正确引用误判为「超出片段」。把裁判上下文改成完整原文并对齐编号后，Q5 恢复 2 分。

Q15 是当前唯一真实语料空白：满减与店铺券能否叠加。RAG 组如实回答「知识库未覆盖」并给出相关引用，行为正确但不给确定规则；无 RAG 组凭预训练常识给出平台通用规则，裁判给 2 分。这题恰好说明 RAG 的价值不在公开常识，而在私有数据、新鲜数据、防编造与可溯源。

4.4.5 检索质量自测

`python -m backend.app.rag.selftest` 跑 10 条查询，覆盖 10 个类目，判定标准是 top-3 中至少出现一次期望类目。

指标	结果	阈值
通过条数	**9 / 10**	≥ 8 / 10
top-3 相关率	**90%**	≥ 80%
未通过项	1 条，类目边界模糊导致 top-3 落在相邻类目	记录，不隐藏

90% 是检索层的自测，不是端到端答案准确率。它证明混合检索在 43 篇语料上可用，不证明所有回答都正确；端到端质量由 v1 与 v2 两套评测共同约束。

4.5 多模态

多模态有两条链路：商品图理解与语音输入。两条都属于输入侧增强，不改变 Agent 与 RAG 的主链路。

能力	实现	输入	输出	降级
商品图文案	`POST /v1/vision/caption` → GLM-4V-Flash	`image_url` 或 `image_base64`	title、selling_points、description	未配 key 返回 400 中文提示，主链路不受影响
语音输入	浏览器 Web Speech API	麦克风，`zh-CN`	文本写入输入框	浏览器不支持时按钮隐藏或提示

视觉端点的提示词要求返回 JSON，字段固定为 title、selling_points、description。GLM-4V 有时会把 JSON 包在 ``` ```json 代码块里，解析时用正则提取第一个 ``` 块；解析失败则返回原文，前端把原文显示在识别结果卡里，不阻塞用户继续编辑。识别结果中的标题会自动回填输入框，用户可以改两个字再发送。

语音走浏览器原生识别，零服务端成本。它解决的是移动端与快捷输入，不参与工具调用，也不做声纹或情绪识别。

4.6 MCP 工具暴露

MCP Server 是独立进程，用 FastMCP 的 stdio 传输，启动命令为 `python -m backend.mcp_server`。它从 `backend/app/agent/tools` 的注册表导入实现，暴露 4 个工具：`kb_search`、`data_query`、`copywriting`、`policy_check`。

工具	参数	复用实现
`kb_search`	query	RAG 混合检索
`data_query`	query	`DataProvider` 经营快照
`copywriting`	product_name, selling_points	文案生成
`policy_check`	scenario	检索 + 风险分级

同一份工具实现同时服务两个场景：HTTP 场景下由 ReAct 循环调用，生态场景下由任意 MCP 客户端调用。新增工具只需在注册表注册一次，两个通道同时获得能力。验证命令是 `npx @modelcontextprotocol/inspector python -m backend.mcp_server`，Inspector 中可直接调用 `kb_search` 并看到检索结果。

4.7 AI 安全与可控性

安全设计的核心是**不信任模型传参**。模型可以决定「调哪个工具」，但不能决定「校验是否通过」。所有业务规则在后端执行，模型只负责把预览复述给用户并征求确认。

控制点	机制	失败时行为
资金操作	退款、改址、拦截、退换货统一 validate → execute	校验不过返回 `violations`，不签发令牌
确认	一次性令牌，10 分钟有效，订单与金额锁定	过期、复用、订单不匹配一律拒绝
幂等	已退款订单重复 execute 返回既有回执	不重复退款，不重复建单
审计	售后单号、订单状态、金额、时间落库	可按订单号回查
引用	`[n]` 角标 + `/v1/rag/search` 同源回查	无覆盖时明确说「知识库未覆盖」
密钥	仅存 `backend/.env`，已 gitignore	缺 key 返回 400 中文说明，页面不白屏
参数	JSON Schema 校验	缺参、多参、未知工具返回可读错误
错误	上游错误转中文 JSON，工具异常转字符串回填	SSE 不中断，模型可自我修正

两段式门控的规则表如下。四类操作共享同一套骨架，差异只在业务校验函数。

操作	第一段校验	第二段落库	幂等键
退款	订单存在、已付款、未退过、金额 ≤ 实付、7 天售后期内	更新订单与售后单，令牌作废	订单号 + 已退款状态
改地址	订单存在、已付款、未发货	更新收货地址	订单号 + 地址版本
拦截物流	订单存在、已发货在途、每单限一次	标记拦截，写售后记录	订单号 + 拦截状态
退换货	订单存在、已确认收货、7 天退货期内	生成退换单据，状态「待寄回」	订单号 + 单据号

4.8 优化策略

优化集中在五处：分块、融合、缓存、上下文裁剪、流式与成本。每一条都对应一个具体问题，不做无证据的性能承诺。

优化点	做法	解决的问题	可验证结果
分块	800 字符、200 重叠、按段落边界 flush	规则条文被切断，检索只命中半条	43 篇语料可重建索引
RRF	两路各取 2k 候选，按排名融合	单通道漏召回，向量与 BM25 量纲不一致	自测 9/10 = 90%
缓存	`Retriever` 进程内单例；切片保存后 `invalidate`	每次请求重复加载索引与 BM25 统计	知识库保存后下一次检索即生效
上下文裁剪	`kb_search` 每片段 400 字；步骤摘要 120 字；`order_lookup` 保留 4000 字	tool message 膨胀，多步循环挤爆上下文	10 工具、5 步上限内稳定
流式	工具阶段非流式解析，最终答案按 8 字符块推送	既要完整 `tool_calls`，又要逐字呈现	实测 138 段 delta
成本	本地嵌入、免费视觉档、步数上限、缺 key 降级	API 费用与失控循环	无密钥也能启动与演示非 AI 页面

延迟方面只做结构性优化，不写未测量的毫秒数。检索在请求进入上游前完成，命中才注入；Agent 每轮工具调用都是短请求，最终答案才流式；前端在等待首字时先渲染步骤卡，让用户看到「正在查什么」，而不是空白转圈。这些做法改善的是感知等待，不是伪造吞吐。

---

5 核心功能实现

5.1 多 Agent 编排与客服对话

客服对话是核心入口。页面顶栏是会话标题与新建会话按钮，下方是消息流。每条 Agent 回复上方有「🧠 Agent 处理过程」面板，展开可见 router→售前/售后/投诉→终审 的完整思维链；投诉场景额外显示工单卡（工单号 / 类别 / 紧急度）。输入框上方是 4 张客户场景卡：咨询商品、查订单、退款退货、投诉建议，点击预填问题。消息支持 Markdown、`[1][2]` 角标溯源、复制与重新生成，会话栏支持新建、改名与两步确认删除。

LangGraph 编排在 `backend/app/agent/graph.py`：`intent_router` 用 json_object 强制输出 `{intent, confidence}`，分流到 presales / aftersales / escalation / clarify 四个节点，统一汇入 `review` 终审。售前工具集为 product_lookup + kb_search，售后为 order_lookup / refund_flow / modify_address / intercept_logistics / return_exchange + kb_search，**职责隔离**。终审做两道硬校验：无工具证据强制拒答；有知识库命中但答案缺 `[n]` 记警告日志（幻觉防御）。

数据来自 `DataProvider` 查询种子库。当前演示数据里，最新一天退款率 3.13%，高于 2.5% 告警线；库存告急两项：儿童保温杯 2 件、桌面收纳盒 4 件。

5.3 数据

数据页提供订单、商品、售后三张只读表，支持搜索、状态筛选与结果计数。行内有两个动作：「对话」跳转到 `/chat` 并预填订单号；「退款」直接打开 `RefundDialog`，走同一套 `/v1/refund/validate` 与 `/v1/refund/execute`。这样即使模型在某一轮没有调用工具，用户仍有一条确定的办理路径。数据页不是 Agent 的替代品，而是它的兜底与核对界面。

5.4 知识库

知识库页展示 43 篇文档与切片列表。点开文档展开切片，正文在 textarea 中可直接编辑。保存时 `PUT /v1/kb/chunks/{id}` 做三件事：重新嵌入该切片、写回 LanceDB、让检索单例失效。下一次提问会重新加载索引，读到新文本，不需要重启服务。

演示路径用《退款率过高的后果与控制》验证：把「类目均值」改成「8%」，保存后回对话问「退款率控制在多少比较安全」，回答出现 8%，`[1]` 仍能溯源到该文档。验证完点「重建索引」，从 `data/raw` 全量重跑，把语料恢复原样。难点不在写入，而在让进程内缓存与 BM25 统计同步失效：`Retriever` 做成单例，保存后 `invalidate_retriever()`，下次 `get_retriever()` 惰性重建。

5.5 两段式退款门控

退款是风险最高的操作，也是「咨询即办理」最需要证明的场景。系统把它拆成两段，模型只负责沟通，后端负责判定。

```
第 1 段  validate
用户：帮我退 80 块
  → refund_flow(action=validate, order_id, amount)
  → 后端规则：存在 / 已付款 / 未退过 / 金额≤实付 / 7 天售后期
  → ok=true：返回 preview + token（10 分钟、一次性）
  → 模型复述预览并停下，等待用户确认

第 2 段  execute
用户：确认
  → refund_flow(action=execute, order_id, token)
  → 校验 token（存在 / 未过期 / 订单匹配）
  → 落库：更新订单与售后单 → 令牌作废
  → 回执：售后单号、金额、剩余、时间
  → 重复提交走幂等分支，只回「已退款」
```

以 001 为例。实付 ¥159.8，用户要求退 80，validate 逐条通过并返回「退款 ¥80，剩余 ¥79.8」与一次性令牌；用户确认后 execute 落库，订单变为已退款，售后单记录金额与时间。

同一套门控在前端 `RefundDialog` 中复用。弹窗三步：输入订单与金额、点「下一步：校验」、确认预览后点「确认退款」。聊天内工具调用与弹窗调用命中同一组后端接口，规则不会出现两套。

校验失败同样有明确出口。场景 4 中用户要求 001 退 418，validate 返回 `ok=false`，`violations` 为「退款金额 418.0 超过实付金额 159.8」，不签发令牌。模型原样转述拒因，并给出两条合规替代：按实付 159.8 整单退，或走价保补差价。

5.6 订单卡与结构化工具结果

`order_lookup` 返回结构化 JSON：`{ok, order_count, orders[], aftersales_count, aftersales[]}`。Agent 步骤卡解析 `orders[0]`、`order` 或顶层 `order_id`，渲染成订单卡，展示订单号、买家、商品、实付、状态、物流方与运单号。009 的卡片显示郑*、不锈钢保温饭盒 ×2、¥178、已发货、圆通速递 YT9182736455102。

订单卡右侧有「处理退款」按钮，点击打开 `RefundDialog` 并预填订单号。这条链路把「查询结果」直接变成「下一步动作」，用户不用记住订单号再手输。

5.7 引用溯源

引用链路的关键是「生成用什么，回查就返回什么」。生成端调用 `retrieve(query)` 得到 top-4，按 RRF 顺序编号注入 system；回查端 `GET /v1/rag/search?q=<原问题>&k=4` 调用同一个 `retrieve`，同一个 `top_k`，同一个排序。因此 `[2]` 一定对应生成时看到的第 2 个片段，不会出现「生成时是 A 文档，点开变成 B 文档」。

前端 `MarkdownText` 把 `[1][2]` 预处理成可点角标，点击后 `CitationCard` 在右下弹出，显示标题、分类、来源与原文片段。角标只在有检索上下文时出现；没有命中的回答不会伪造引用。

5.8 流式渲染与会话状态

流式渲染分两段。Agent 的工具阶段不流式，因为要完整解析 `tool_calls`；最终答案按 8 字符块发 `delta`，前端逐块追加。RAG 路径直接把上游 SSE 字节流转发，前端 `useChat` 消费。两条路径在 UI 上表现一致：步骤卡先于文字出现，文字逐字出现。

会话切换有四层防护：debounce 1.5s 延迟 `PUT` 全量覆盖并在切换前 flush；`signature(id, messages)` 与 `savedSigRef` 比对，相同则跳过；`loadingRef` 在加载窗口内禁止 flush；`activeIdRef` 让回调闭包读 ref 而非 state。会话 API 用 `PUT /messages` 覆盖消息，`PATCH` 改标题或模式，`DELETE` 删除；删除是两步确认，自动标题在首条消息发送时写入。

---

6 测试与部署

6.1 测试矩阵

测试分六层：静态检查、构建、接口冒烟、检索与效果评测、浏览器验收、关键资金路径。每层都有可执行命令或可复核证据，不靠主观描述。

层次	内容	命令或方法	结果
静态检查与构建	tsc、ESLint、next build	`tsc --noEmit`、`lint`、`build`	三项退出码 0
接口与会话	冒烟 8 接口 + 4 页面；会话 API 5 项	`smoke.mjs`、`p4_api_test.py`	11/11；12/12
检索质量	10 条查询 top-3	`python -m backend.app.rag.selftest`	**9/10 = 90%**
RAG 效果	20 题 + 15 题对照	`eval.py`、`eval_v2.py`	26 vs 23；29 vs 30
MCP	Inspector 调 4 工具	`npx @modelcontextprotocol/inspector`	4 tools 可用
浏览器验收	4 页 × 1440/390	Playwright，F3 报告	无横向滚动，console error 为空
资金路径	退款校验、确认、回执	001 退 80，确认页与回执	剩余 ¥79.8，未重复退款

6.2 结果说明

`tsc`、`eslint`、`next build` 退出码均为 0。冒烟 11/11 覆盖 7 个接口与 4 个页面，脚本对 `/v1/intent` 的 `{message}` 与 `{q}` 契约差异做了兼容。检索自测 9/10，未通过的一条是类目边界模糊；端到端评测 v1 为 26 比 23，v2 为 29 比 30。公开常识题裸模型已很强，RAG 的增量在「有据可查、无据敢说不知道、私有数据可注入」。

6.3 浏览器端验收

F3 用 Playwright 在 1440×900 与 390×844 两个视口验收 4 个页面。首轮发现工作台横向滚动、待办缺买家与商品、退款确认页金额空白、聊天内订单卡不渲染四类问题。修复后终检结论为 APPROVE。

验收项	结果	证据
工作台 1440 与 390 无横向滚动	通过	`scrollWidth == clientWidth`
待办 4 件，含买家与商品	通过	016、017、011、013 四行
退款确认页金额可见	通过	「退款 ¥80，剩余 ¥79.8」
订单卡与「处理退款」按钮	通过	009 卡片免展开，按钮计数 1
四页 console error	通过	均为空数组
知识库保存与还原	通过	8% 生效后还原为类目均值

6.4 部署步骤

项目按双服务部署。Python 服务提供 AI 能力，Next.js 提供界面与同源代理。两个服务都在项目根目录启动，`backend` 作为 Python 包被导入。

```
# 终端 1：Python AI 接入层
pip install -r backend/requirements.txt
Copy-Item backend/.env.example backend/.env   # 填入 DEEPSEEK_API_KEY
python -m backend.app.agent.seed_store        # 生成 30 天种子库（幂等）
python -m backend.app.rag.pipeline build      # 构建 LanceDB 索引
uvicorn backend.app.main:app --port 8000

# 终端 2：Next.js 前端
npm install
npm run build
npm run start                                  # 生产模式，默认 3000
```

打开 `http://localhost:3000`，根路径自动跳转 `/chat`。开发模式用 `npm run dev`。`data/index`、`data/app.db`、`data/store.db`、`backend/.env` 都不入库，首次运行需按上面命令生成。

6.5 部署所需的最小变量集

部署最少只需配置 `DEEPSEEK_API_KEY`；`BACKEND_URL`、`AI_BASE_URL`、`DATA_PROVIDER`、`VISION_API_KEY` 均有默认值或可留空。完整变量、默认值与降级行为见附录 B。

6.6 降级与容错

场景	系统行为	用户看到
缺 `DEEPSEEK_API_KEY`	聊天与 Agent 返回 400	中文提示「请在 backend/.env 配置后重启后端」
缺索引或 `store.db`	`retrieve` 返回空；`DataProvider` 回退 mock	正常聊天；数据页样例数据标记 mock
缺 `VISION_API_KEY`	视觉端点返回 400	图片识别失败提示，文本与 Agent 不受影响
上游限流或 502	返回带中文说明的 JSON	消息内联错误，页面不白屏
超过 5 步	返回明确错误事件	「已达最大工具调用步数，请简化问题」

6.7 已知问题与处置

问题	影响	处置
长会话中模型偶发不调工具，直接给文字结论	退款演示可能不出现预览卡	`RefundDialog` 提供确定路径；演示用新会话
1440 视口订单详情抽屉覆盖发送按钮	鼠标点击受限，Enter 仍可用	记录为 UX 待修

这些问题是真实存在的，不影响主链路验收，但演示前应知道兜底路径在哪里。

---

7 创新点说明

7.1 技术创新点

T1 两段式工具门控。 把资金操作拆成 validate 与 execute，中间插入用户确认与一次性令牌；模型不持有执行权，后端不信任模型传参，重复提交走幂等。这把「Agent 能不能动钱」从提示词约束变成结构约束。对应评分点：AI 安全可控 4%、Agent/工作流 8%。

T2 单注册表双通道。 10 个工具注册一次，同时服务 ReAct 循环与 MCP Server；新增工具不改循环、不改协议、不改前端。对应评分点：MCP/工具集成 4%、技术方案创新 8%。

T3 同源引用溯源。 生成端与回查端共用同一个检索器、同一组参数、同一种排序，`[n]` 编号在两端一致。对应评分点：RAG 6%、AI 大模型集成 8%。

T4 编辑即生效的知识库。 切片保存后重新嵌入、写回 LanceDB、失效检索单例，下一次提问即读到新文本。对应评分点：RAG 6%、实用性 30%。

7.2 应用创新点

A1 咨询即办理。 对话里查单、退款、改址、拦截、退换货，敏感操作有门控，回执可核对，用户不用在「问」和「做」之间切换系统。对应评分点：选题创新 10%、实用性 30%。

A2 客服对话与知识库双入口闭环。 对话解决客户问题，知识库沉淀商家规则；两个入口共享会话与工具，改完知识库下一句提问即用上新口径。对应评分点：交互创新 7%、前端工程 10%。

A3 投诉处理方案。 复杂投诉不推到系统外，直接在对话里生成情况摘要、建议动作、可复制话术。对应评分点：实用性 30%、选题创新 10%。

A4 结构化结果直接变界面。 工具 JSON 不只喂给模型，还渲染成订单卡、退款预览、引用卡、步骤卡。对应评分点：交互创新 7%、前端工程 10%。

7.3 创新点与评分维度映射

评分维度	权重	对应创新点	可演示证据
选题创新	10%	A1、A3	对话内退款、投诉处理方案
技术方案创新	8%	T2、T3	双通道注册表、同源引用
交互创新	7%	A2、A4	四入口、步骤卡、订单卡
实用性	30%	A1、A2、A3、T4	12 段场景、知识库编辑即生效
AI 大模型集成	8%	T3	流式回答、结构化输出、角标
Agent/工作流	8%	T1、T2	10 工具、5 步循环、两段式
RAG	6%	T3、T4	90% 自测、v1/v2 评测、溯源
多模态	5%	4.5 节	商品图生成文案、语音输入
AI 安全可控	4%	T1	令牌、确认、幂等、审计
MCP/工具集成	4%	T2	Inspector 调 4 工具
前端工程	10%	A2、A4	TS strict、双视口、组件化

8 总结与展望

8.1 成果

项目完成了一个可运行的智能客服 Agent，而不是演示用的聊天页。LangGraph 多 Agent 编排（Router→售前/售后/投诉升级→终审）、2 个入口、43 篇语料、12 段场景、20 单种子订单、售前 2 + 售后 6 个工具、4 个 MCP 工具全部落地；四类敏感操作走同一套两段式门控，模型不直接动钱；RAG 回答带可点角标，检索自测 90%，交付冒烟 9/9，前端构建与类型检查全绿。前端不直连模型，密钥只在 Python 层，工具异常不中断 SSE，缺密钥与缺索引都有降级路径。

8.2 局限

数据是种子数据。 `data_query` 查询 30 天本地种子库，售后操作作用于 20 单种子订单，不接平台真实授权，不触发真实资金流。`DataProvider` 已插件化，但当前没有接真实源。

语料以方法论为主。 43 篇覆盖 10 个类目，数值型平台规则原文仍不足。v2 的 Q15 说明这一点：满减与店铺券叠加规则没有语料，模型只能靠常识。

评测样本小。 v1 20 题、v2 15 题，裁判是 LLM，存在采样波动与误判，绝对分数不宜过度解读。

单机与单用户。 没有账号体系、多租户与云端同步；长会话混合话题时模型偶发跳过工具直接给结论，产品用 `RefundDialog` 与数据页行内按钮兜底。

8.3 路线图

阶段	目标	关键工作	验收
P6	接真实店铺数据	实现 DataProvider 的 ERP/API 分支，保留 mock 兜底	切换 DATA_PROVIDER 后经营数据与订单查询走真实源
P7	提升 RAG 上限	补数值型规则原文，加入重排与检索指标 Hit@k / MRR	语料扩充后 v2 空白题下降，检索指标可报告
P8	长会话可用	按 token 预算裁剪历史，工具结果先剥离，再做摘要	数百轮会话不撞上下文窗口，刷新可恢复
P9	产品化	账号、多租户、权限、审计导出、图表可视化	多店铺隔离，操作可导出，指标可下钻

P8 的长会话压缩目前只是设计储备，未实现；当前采用全量重放，演示规模下简单可靠。

附录

附录 A API 清单

方法	路径	用途
GET	/health	健康检查
POST	/v1/chat/completions	RAG 问答，流式
POST	/v1/agent/chat	Agent SSE
POST	/v1/vision/caption	商品图文案
GET	/v1/rag/search	引用溯源
POST	/v1/intent	意图分类
GET	/v1/workspace/summary	工作台数据
GET	/v1/data/orders、/products、/aftersales	三张只读表
GET	/v1/data/order/{id}	订单详情
GET	/v1/kb/docs	文档列表
GET/PUT	/v1/kb/chunks	切片读写
POST	/v1/kb/rebuild	重建索引
GET/POST	/v1/sessions	会话列表/新建
GET/PATCH/DELETE	/v1/sessions/{id}	恢复/改名/删除
PUT	/v1/sessions/{id}/messages	覆盖消息
POST	/v1/refund/validate、/execute	校验与执行
POST	/v1/escalate	投诉方案

附录 B 环境变量清单

变量	默认值	说明
DEEPSEEK_API_KEY	无	必填，文本模型与 Agent
DEEPSEEK_BASE_URL	https://api.deepseek.com	上游基址
DEEPSEEK_MODEL	deepseek-chat	文本模型名
RAG_ENABLED	true	总开关，请求体 rag 可覆盖
EMBED_PROVIDER	local	local 或 siliconflow
SILICONFLOW_API_KEY	空	仅 siliconflow 时需要

变量	默认值	说明
VISION_API_KEY	空	缺失时视觉端点返回 400
VISION_MODEL	glm-4v-flash	视觉模型名
VISION_BASE_URL	https://open.bigmodel.cn/api/paas/v4	视觉上游基址
DATA_PROVIDER	sqlite	sqlite 真查，mock 兜底
AI_BASE_URL	http://127.0.0.1:8000/v1	前端 provider 基址
BACKEND_URL	http://127.0.0.1:8000	同源代理目标

附录 C 复现命令

```
# 索引与种子数据
python -m backend.app.rag.pipeline build
python -m backend.app.agent.seed_store

# 质量门禁
python -m backend.app.rag.selftest
python -m backend.app.rag.eval
python -m backend.app.rag.eval_v2
npx tsc --noEmit
npm run lint
npm run build
node scripts/smoke.mjs

# MCP
npx @modelcontextprotocol/inspector python -m backend.mcp_server
```

文档版本 v1.2 · 2026-10-28 · 重构为竞赛八章结构，补齐需求分析、AI 选型理由、优化策略、测试与部署、创新点、总结与展望；事实基线更新为 43 篇语料。